Università degli Studi di Modena e Reggio Emilia

Facoltà di Ingegneria – Reggio Emilia

CORSO DI

RETI DI CALCOLATORI Linguaggio Java: La Programmazione di Rete

Prof. Franco Zambonelli

Lucidi realizzati in collaborazione con Ing. Enrico Denti - Univ. Bologna

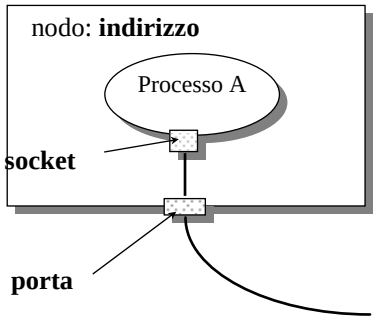
JAVA E LA RETE

L'architettura Java è *network-ready*

- Package `java.net`
  Concetti (classi) fondamentali:
  - `Socket`, `ServerSocket`
  - `URL`, `AudioClip`, `Image`
- Package `java.rmi`
  Uso di *oggetti remoti: si possono richiedere servizi a oggetti che risiedono su altri nodi della rete Internet*
- ...

IL CONCETTO DI SOCKET

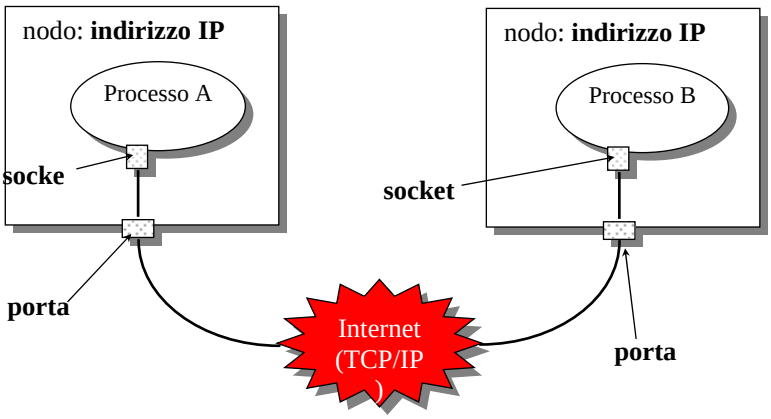
- Un'oggetto *socket* è concettualmente una *porta*, una "*presa*" verso la rete
- Collega un certo *processo* al mondo esterno
- È identificata da un *numero (port number)* unico *su una data macchina* (*nodo* o *host*)
- Ogni nodo è identificato dal suo *indirizzo IP*



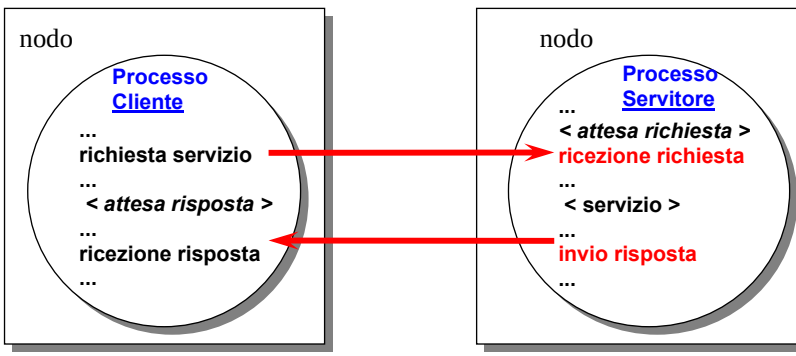
COMUNICAZIONE VIA SOCKET

- La socket è un *canale di comunicazione*
- Permette a due processi, *residenti sulla stessa macchina o anche molto distanti,* di comunicare fra loro *nello stesso modo*
- Tipicamente Modello cliente / servitore:
  - il servitore deve *stare in attesa* di possibili comunicazioni in arrivo *(ente passivo)*
  - i clienti (anche più di uno), quando vogliono, *parlano* con il servitore *(enti attivi)*

COMUNICAZIONE VIA SOCKET



Per realizzare Schemi cliente-servitore



TIPI DI SOCKET

- Esistono fondamentalmente *due tipi di socket:* socket stream e socket datagram

- Le socket stream (per comunicare con TCP)
  - sono ***affidabili,*** stabiliscono una connessione ***stabile*** e ***bidirezionale*** con l'altra parte, che dura finché non si decide di chiuderla

    - Usare quando l'ordine dei messaggi è importante e l'affidabilità è cruciale
    - Spesso c'è un limite massimo alle connessioni che si possono aprire

- Le socket datagram (per comunicare con UDP)
  - *non* sono affidabili, *non* stabiliscono una connessione stabile: la comunicazione è *unidirezionale* come un telegramma
  - ma sono *meno costose*

    - Usare quando le prestazioni sono fondamentali e/o ci vorrebbero troppe connessioni aperte insieme
    - Non devono esserci problemi se i messaggi arrivano in ordine qualunque

SCHEMA COMUNICAZIONE CON SOCKET STREAM (TCP)

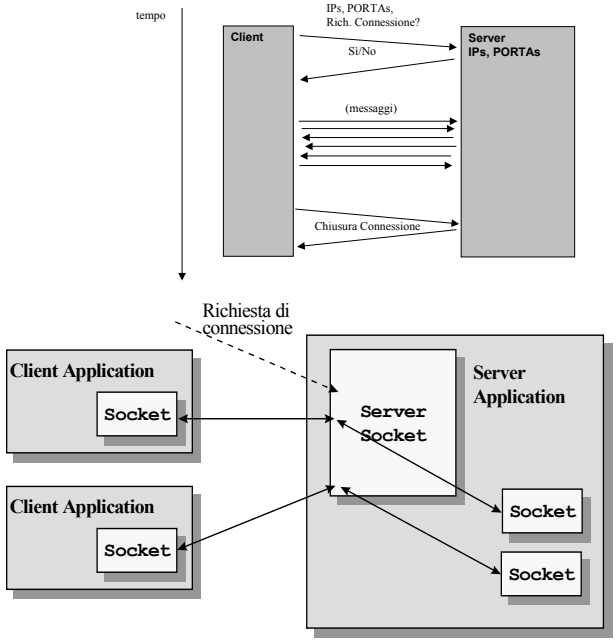
1) Il servitore crea la sua `ServerSocket` con un *numero noto,* e si mette in attesa
2) Un cliente, quando vuole comunicare col servitore, crea la sua `Socket` *specificando con chi vuole parlare*
   - nome dell'host
   - numero di porta

   questo fa scattare il protocollo di **3-way handshaking** per la connessione
3) Il servitore accetta la richiesta del cliente: con ciò si stabilisce una connezioe e si il server crea una nuova `Socket` *già collegata al cliente,* tramite cui i due comunicano.

A questo punto, i processi possono parlare senza più bisogno di specificare ogni con chi si vuole parlare (come nelle telefonate…una volta presa la linea, tutti i segnali transitano automaticamente tra le due parti…)

- Alla `Socket` sono associati *due stream*:
  - uno *dal cliente verso il servitore*
  - uno *dal servitore verso il cliente*
- La comunicazione cliente/servitore è *bidirezionale*
  - i ruoli "cliente" e "servitore" *sono tali solo nella fase iniziale*, quando si instaura la connessione
  - una volta connessi, i due processi si parlano reciprocamente "alla pari"

SCHEMA CON SOCKET STREAM



La `ServerSocket` serve per *stare in attesa di richieste dai clienti*: quando ne arriva una, *l'effettiva comunicazione avviene tramite una nuova `Socket`* appositamente creata

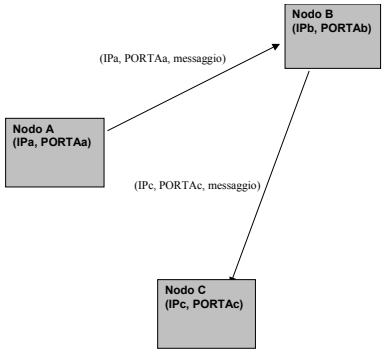
SCHEMA COMUNICAZIONE CON DATAGRAM

1) Il servitore crea la sua `DatagramSocket` con un *numero noto,* e si mette in attesa
2) Un cliente crea la sua `DatagramSocket`
3) Quando vuole inviare un messaggio al servitore, il cliente *gli manda un "data-gramma"*
   - nome dell'host e numero di porta

Non si crea *alcuno stream stabile*
- comunicazione solo *dal cliente al servitore*
- ogni dato inviato deve specificare a "chi" deve essere inviato (come per nelle corrispondenze via lettera)

In effetti si perde un po' la distinzione tra cliente e servitore.

Le comunicazioni datagram sono comunicazioni tra "pari".



LA CLASSE `Socket`

Costruire una `Socket` significa *aprire la comunicazione* verso l'altra parte

- `public Socket(String remoteHost, int remotePort)`
  - **Crea una socket stream e la collega alla porta specificata della macchina remota corrispondente al nome dato**

Esempio

```
Socket s = new Socket("www.nasa.gov",8080);
```

Oppure

```
String ss = "prova.com"; int port;
Socket s = new Socket(ss,port);
```

NOTA: non bisogna specificare il nome dell'host locale, è implicito. Per il client, non bisogna specificare il numero della porta (non dev'essere noto a priori). Il sistema assegna automaticamente un numero di porta tra quelli liberi.

NOTA: il servizio di DNS è invocato automaticamente

– (segue) LA CLASSE Socket

Per comunicare, *si recuperano dalla socket i due stream* (di ingresso e di uscita) a essa associati, tramite i metodi:

- `public InputStream getInputStream()`
  - **restituisce lo stream di input da cui leggere i dati (byte) che giungono dall'altra parte**
- `public OutputStream getOutputStream()`
  - **restituisce lo stream di output su cui scrivere i dati (byte) da inviare all'altra parte**

Poi, su questi stream si legge / scrive *come su qualunque altro stream di byte.*

Al termine, *per chiudere la comunicazione*

si chiude la `Socket` col metodo:

- `public synchronized void close()`
  - **chiude la connessione e libera la risorsa**

La parola chiave `synchronized` ***non è rilevante*** **in questa sede (comunque vedi lucidi sui thread!).**

La classe `InetAddress`

La classe InetAddress fornisce delle funzioni (**Metodi di Classe**) per gestire facilmente gli indirizzi Internet. In pratica, un oggetto InetAddress gestiscono, in modo trasparente gli indirizzi nel formato IP a 4 byte.

```
InetAddress.getByName(String);
```

Ritorna un oggetto della classe InetAddress che server per rappresentare indirizzi Internet. Gli oggetti della classe InetAddress rappresentano *indirizzi internet.*

Quindi, per esempio, per ottenere l'InetAddress di un computer dato il suo nome:

```
Inetddress ina =
   InetAddress.getByName("www.ciao.it");
```

ESEMPIO: Per lavorare senza la rete (p.e., da casa), possiamo ottenere gli inetAddress in questo modo

```
InetAddress.getByName(null);
InetAddress.getByName("localhost");
InetAddress.getByName("127.0.0.1");
```

Ovviamente, quando abbiamo un InetAddress, possiamo fare l'operazione inversa, e cioè recuperare il nome:

```
String nome = ina.getHostName();
```


Usare la classe `InetAddress` nelle Socket

Alcuni metodi utili della classe socket maneggiano InetAddress:

- `public InetAddress getInetAddress()`
  - **restituisce l'indirizzo della macchina remota a cui la socket è connessa**
- `public InetAddress getLocalAddress()`
  - **restituisce l'indirizzo della macchina locale**

Quindi, da una socket aperta, possiamo ottenere informazioni sui nodi corrispondenti, cioè sugli indirizzi dei due host cui essa è connessa

E poi, naturalmente, possiamo anche conoscere i numeri di porta della connessione:

- `public int getPort()`
  - **restituisce il numero di porta sulla macchina remota a cui la socket è connessa**
- `public int getLocalPort()`
  - **restituisce il numero di porta sulla macchina locale a cui la socket locale è associata**

Si può una socket anche in questo modo:

- `public Socket(InetAddress remoteAddr, int remotePort)`

Crea una socket stream e la collega alla porta specificata della macchina remota corrispondente all'indirizzo IP dato

SERVIZI STANDARD

Moltissimi servizi di uso comune sono *standardizzati* e disponibili su macchine sia Unix sia (non sempre) Windows

- echo (porta 7): **rimanda indietro tutto quello che gli si invia, come un'eco**
- daytime (porta 13): **restituisce data e ora**
- telnet (porta 23): **consente il collegamento remoto, da altro terminale (solo Unix)**
- smtp (porta 25): **consente la spedizione della posta (se abilitata)**
- http (porta 80): **protocollo http per comunicazione tra browsera e www server**

Nel caso, provare a eseguire gli esercizi che seguono collegandosi non sul localhost (che tipicamente è Windows e non ha questi servizi) ma collegandosi verso una macchina UNIX (p.e., polaris.ing.unimo.it)

ESEMPIO 1

Un cliente che si connette a una macchina remota sulla porta di daytime (porta 13), recupera data e ora, e li visualizza.

SCHEMA ESEMPIO:

```
import java.net.*;
import java.io.*;
public class EsempioNet1 {
 public static void main(String args[]){
  // creazione socket verso porta 13
  // recupero stream di input
  // lettura del messaggio dallo stream
  // stampa a video del messaggio
 }
}
```


ESEMPIO 1

```
import java.net.*;
import java.io.*;
public class EsempioNet1 {
 public static void main(String args[]){

  try { Socket s =new Socket("pippo.com",13);
// ATTENZIONE: la apertura di una socket può
// generare eccezione ConnectException
// se il server non esiste o rifiuta il servizio
    InputStream is = s.getInputStream();
    InputStreamReader ir =
   new                InputStreamReader(is);
    BufferedReader r =
             new BufferedReader(ir);

    String line = r.readLine();
    System.out.println(line);
    s.close();
    } catch (UnknownHostException e){
       System.err.println("Host unknown");
    } catch (Exception e){
       System.err.println(e);
    }  }
}
```

RISULTATO: Fri Feb 18 16:44:46 2000

ESEMPIO 2

Un cliente che si connette a una macchina remota sulla porta di echo (porta 7), le invia un messaggio, e stampa ciò che riceve.

SCHEMA ESEMPIO:

```
import java.net.*;
import java.io.*;
public class EsempioNet2 {
 public static void main(String args[]){
  // creazione socket verso porta 7
  // recupero stream di input e di output
  // invio messaggio su stream di output
  // lettura e stampa da stream di input
 }
}
```

ESEMPIO 2

```
import java.net.*;
import java.io.*;
public class EsempioNet2 {
 public static void main(String args[]){
  Socket s = null;
  try { s = new Socket("ing.unimo.it",7);
    InputStream is = s.getInputStream();
    InputStreamReader ir =
          New InputStreamReader(is);
    BufferedReader inr =
            new BufferedReader(ir);

   OutputStream os = s.getOutputStream();
   OutputStreamWriter or =
          new OutputStreamWriter(os);
    BufferedWriter outr =
          new BufferedWriter(or);

    String msg = "Hello, I'm Donald Duck!";
// oppure potrei leggere la linea da input…
   outr.write(msg, 0, msg.length());
   outr.newLine();//invia fine lina
   outr.flush(); //svuota buffer
    //istruzioni imprtanti
```

(continua)

ESEMPIO 2 (continua)

```
   String line = inr.readLine();
   System.out.println("Ricevuto: ");
   System.out.println(line);
   s.close();
   } catch (UnknownHostException e){
      System.err.println("Host unknown");
   } catch (Exception e){
      System.err.println(e);
   }  }}
```

Risultato Esempio 2:
`Hello, I'm Donald Duck!`

NOTA IMPORTANTE:

Le operazioni di scrittura e lettura su uno stream di socket sono **BLOCCANTI**:

- si può leggere solo se ci sono dati che sono stati inviati, altrimenti la esecuzione si sospende fino a quando non arrivano i dati
- la operazione di scrittura su uno stream di socket blocca la esecuzione del processo che scrive fino a quando i dati non sono stati ricevuti e fino a quando non ritornano indietro gli ack dei messaggi ricevuti

UN SERVITORE

Un servitore deve creare la propria `ServerSocket` *su una porta predeterminata*

- il numero di porta del server *deve essere noto ai clienti* perché ne avranno bisogno per connettersi al servitore
- per i servizi standard, i numeri di porta sono standardizzati
- per i nostri servizi, possiamo scegliere un numero qualunque, purché libero (1-1024 sono riservati)

Costruire una `ServerSocket` significa *predisporsi a ricevere richieste di connessione* da clienti remoti.

- `public ServerSocket(int localPort)`
  - **Crea una ServerSocket e la collega alla porta locale specificata**
- `public ServerSocket(int localPort, int count)`
  - **Crea una ServerSocket che accetta al massimo count richieste pendenti, e la collega alla porta locale specificata**

FARE ATTENZIONE CHE LA PORTA LOCAL NON SIA RISERVATA O GIA' IN USO!!!

Esempio

```
ServerSocket s = new ServerSocket(13000);
```


LA CLASSE ServerSocket

Il metodo principale

- `public Socket accept()`
  - mette il servitore in **attesa** di nuove richieste di connessione: se non ce ne sono, il servitore si blocca in attesa
  - restituisce un oggetto `Socket`, tramite cui avviene l'effettiva comunicazione tra cliente e servitore

NOTE IMPORTANTI!!!
La esecuzione del servitore si **BLOCCA** sulla esecuzione della accept fino a questo non arriva la richiesta di qualcuno che vuole connettersi.
Quando arriva la richiesta di connessione dal cliente (e viene eseguito il protocollo 3-way handshaiking) la esecuzione prosegue e si ha a disposizione una nuova socket per comunicare.
D'ora in poi, anche il servitore comunica col cliente tramite una normale `Socket`. Valgono quindi gli stessi mezzi visti poc'anzi.
La server socket rimane a dispozisione per ulteriori richieste di connessione

Altri metodi utili

- `public InetAddress getInetAddress()`
  - **restituisce l'indirizzo della macchina locale**
- `public int getLocalPort()`
  - **restituisce il numero di porta sulla macchina locale a cui la socket è legata**

TIPICO SCHEMA DI SERVER

```
...
 ServerSocket ss = new ServerSocket(porta);
 try {
  while (true) {
/*una volta attivato il server non termina
 mai; svolge per definizione un ciclo
 infinito */
    Socket clientSock = ss.accept();
   // clientSock è nuova socket tramite cui
   // si parla col cliente
     ... fase di comunicazione col cliente...
   clientSock.close();
  }  } catch (Exception e) { ... }
```


ESEMPIO DI SERVER: daytime

Un nostro servitore daytime sulla porta 7777 (rimanda indietro la data corrente)

SCHEMA DEL SERVER:

```
import …;
public class EsempioServer {
 public static void main(String args[]){
  // creazione ServerSocket su porta 7777
  // ciclo senza fine:
  //  attesa connessione,
  //  recupero socket cliente,
  //  recupero stream e comunicazione
  //  chiusura socket cliente
 }
}
```


ESEMPIO DI SERVER daytime (codice)

```
import java.net.*;
import java.io.*;
public class EsempioServer {
 public static void main(String args[]){
  ServerSocket ss = null;
  try {
   ss = new ServerSocket(7777,50);
   while (true) {
    Socket clientSock = ss.accept();
     System.out.println("cliente connesso")
    OutputStream os =
         clientSock.getOutputStream();

   PrintStream outp = new PrintStream(os);
    outp.println(new java.util.Date());
    clientSock.close();
    }
  } catch (UnknownHostException e){
    System.err.println("Host unknown");
  } catch (Exception e){
     System.err.println(e);
  }
 }
}
```


ESEMPIO DI SERVER daytime (codice)

GLI FACCIAMO RIFIUTARE CONNESSIONI DA CLIENTI CHE NON CI PIACCIONO

```
import java.net.*;
import java.io.*;
public class EsempioServer {
 public static void main(String args[]){
  ServerSocket ss = null;
  try {
   ss = new ServerSocket(7777);
   while (true) {
    Socket clientSock = ss.accept();

    InetAddress cina = ss.getInetAddress();
    String shn = cina.getHostName();
    If (shn.equals("www.hostantipatico.it")
         {// chiudiamo la connessione
         ss.close();
    else { // procediamo normalmente
         System.out.println("cliente connesso")
         OutputStream os =
              clientSock.getOutputStream();
          PrintStream outp = new PrintStream(os);
          outp.println(new java.util.Date());
          clientSock.close();
         }
     }
  } catch (UnknownHostException e){
    System.err.println("Host unknown");
  } catch (Exception e){
     System.err.println(e); }
 }}
```

ALTRO ESEMPIO: il server di ECHO

```
import java.net.*;  import java.io.*;

public class EchoServer {
public static void main(String args[]){
    Socket cs = null; ServerSocket ss = null;

String msg;
    try {
     ss = new ServerSocket(7779);
     while (true) { // ciclo infinito
      cs = ss.accept();

       InputStream is = cs.getInputStream();
     InputStreamReader ir =
      new InputStreamReader(is);
    BufferedReader r =                  new
            BufferedReader(ir);

   // recupera e sistema output stream socket
     OutputStream os = cs.getOutputStream();
     OutputStreamWriter or =
                 new OutputStreamWriter(os);
     BufferedWriter outr =
                 new BufferedWriter(or);

   // legge una linea da socket
   msg = r.readLine();

    // scrive la stringa sulla socket
     outr.write(msg, 0, msg.length());
     outr.newLine();//invia fine lina
     outr.flush(); //svuota buffer
        //istruzioni imprtanti

    // legge una linea dalla socket

        os.close(); is.close(); cs.close();
   } // end while
  } catch (UnknownHostException e){
      System.err.println("Host unknown"); }
   catch (Exception e){ System.err.println(e); }
 } }
```


CLIENT & SERVER: USO

Se ora usiamo il clienti visti prima con i nostro server:

```
public class EsempioNet2 {
 public static void main(String args[]){
  Socket s = null;
  try { s = new Socket("localhost",7777);
  ...
```

UN ESEMPIO "CLIENT + SERVER"

- Il server risponde con un numero progres-sivo, a partire da 1, a ogni cliente che si connette.
- Il cliente si limita a visualizzare tutto quello che gli arriva dal server, fino a quando non riceve il messaggio `Stop`: a quel punto, il cliente termina.

Il cliente

```
import java.net.*;  import java.io.*;
public class Cliente1 {
 public static void main(String args[]){
  Socket s = null;
  try {
   s = new Socket("localhost",11111);
   BufferedReader r =
    new BufferedReader( new
    InputStreamReader(s.getInputStream()));
   String line;
```


Il cliente (segue)

```
   ...
   while((line=r.readLine())!=null ){
    System.out.println(line);
    if (line.equals("Stop")) break;
   }
   r.close(); s.close();
  } catch (UnknownHostException e){
     System.err.println("Host unknown");
  } catch (Exception e){
     System.err.println(e);
  }  } }
```

Il server

```
import java.net.*;  import java.io.*;
public class Server1 {
 public static void main(String args[]){
  Socket cs = null; ServerSocket ss = null;
  int numero = 1;
  try {
   ss = new ServerSocket(11111);
   while (true) { // ciclo infinito del
  server
    cs = ss.accept();
    System.out.println("Cliente conesso mando" +
numero) ;
   ...
```


Il server (segue)

```
   PrintWriter pw = new
     PrintWriter(cs.getOutputStream(),true);
   // il parametro true significa
   // autoflush attivo
   pw.println("Nuovo numero: " + numero);
   numero++; pw.println("Stop");
   pw.close(); cs.close();
    } // end while
   } catch (UnknownHostException e){
     System.err.println("Host unknown"); }
    catch           (Exception           e){
 System.err.println(e); }
 } }
```

Il risultato invocando più clienti:

```
D:\esercizi>java Cliente1
Nuovo numero: 1
Stop
D:\esercizi>java Cliente1
Nuovo numero: 2
Stop
D:\esercizi>java Cliente1
Nuovo numero: 3
Stop
```

UN ALTRO ESEMPIO client/server

Una mini-calcolatrice “client / server”

- Il cliente invia al server una serie di valori `double`: lo 0 indica la fine della sequenza **(ipotesi: i valori sono sulla riga di comando).** Poi si mette in attesa del risultato dal server, e lo stampa a video.
- Il server riceve dal cliente una serie di valori `double`, e *li somma uno all’altro* fino a che riceve uno 0; a quel punto, invia il risultato al cliente.

UN ALTRO ESEMPIO client/server: Il cliente

```
import java.net.*;  import java.io.*;
public class Cliente2 {
 public static void main(String args[]){
  Socket s = null;
  try {
   s = new Socket("localhost",11111);
   DataInputStream is =    new
    DataInputStream(s.getInputStream()));
   DataOutputStream os =  New
    DataOutputStream(s.getOutputStream()));

   for (int i=0; i<args.length; i++) {
    System.out.println("Sending " + args[i]);

  os.writeDouble(Double.parseDouble(args[i]))
  ;  }
   os.writeDouble(0);
   // lettura risultato dal server
   double res = is.readDouble();
   System.out.println("Risultato = " + res);
   is.close(); os.close(); s.close();
  }             catch             (Exception
  e){System.err.println(e);}
  } }
```


UN ALTRO ESEMPIO client/server: Il server

```
import java.net.*;  import java.io.*;
public class Server2 {
 public static void main(String args[]){
  Socket cs = null; ServerSocket ss = null;
  try {
   ss = new ServerSocket(11111);
   while (true) { // ciclo infinito
    cs = ss.accept();
    DataOutputStream os =  new
   DataOutputStream(cs.getOutputStream());
DataInputStream os =   new
    DataInputStream(cs.getInputStream());
   double res = 0;
   while(is.available()>0){
    // finche ci sono dati disponibili
    res += is.readDouble();
    os.writeDouble(res);
    os.close(); is.close(); cs.close();
   } // end while
  } catch (UnknownHostException e){
   System.err.println("Host unknown"); }
   catch           (Exception           e){
  System.err.println(e); }
 } }
```


ESEMPIO client/server: Il risultato in alcune situazioni:

```
D:\esercizi>java Cliente2 3 4 5
Sending 3
Sending 4
Sending 5
Risultato = 12.0
D:\esercizi>java Cliente2 34 5
Sending 34
Sending 5
Risultato = 39.0
```

ESEMPIO: TRASMISSIONE DI OGGETTI

Il cliente crea un oggetto contatore (come quello che conosciamo già con la accortezza di fargli implementare la interfaccia serializable)

Il servitore riceve l'oggetto, lo fa incrementare, e poi lo rimanda indietro al cliente.

NOTA: Per questo esercizio come per tutti gli altri è necessario stabilire il protocollo di comunicazione.

Per esempio: se sia il cliente che il servitore tentano di leggere qualcosa il programma si blocca!!!!

Per la classe Counter:

```
Import java.io.*;
Public class Counter implements Serializable
…
```


IL CLIENTE

```
import java.net.*;  import java.io.*;

public class ClienteCC {
 public static void main(String args[]){
  Counter c = new Counter();
    Socket s = null;
    c.inc();
System.out.println("Valore Contatore: " +
c.rd());
  try {
   s = new Socket("localhost",11112);
   ObjectInputStream is =    new
    ObjectInputStream(s.getInputStream());
   ObjectOutputStream os =  new
    ObjectOutputStream(s.getOutputStream());

  os.writeObject(c);
   c = (Counter)is.readObject();
System.out.println("Valore Contatore: " +
c.rd());
os.close();is.close(); s.close();
  }
  catch (Exception e){System.err.println(e);}
 }
}
```


IL SERVER

```
import java.net.*;  import java.io.*;

public class ServerCC {
 public static void main(String args[]){
  Socket cs = null; ServerSocket ss = null;
   Counter c;
  try {
   ss = new ServerSocket(11112);
   while (true) { // ciclo infinito
    cs = ss.accept();
    ObjectOutputStream os =   new
   ObjectOutputStream(cs.getOutputStream());
    ObjectInputStream is = new
     ObjectInputStream(cs.getInputStream());
   C = (Counter)is.readObject();
    System.out.println("Ricevuto contatore
con valore: " + c.rd());
    c.inc()
       os.writeObject(c);
        os.close(); is.close(); cs.close();
   } // end while
  } catch (UnknownHostException e){
    System.err.println("Host unknown"); }
   catch           (Exception           e){
 System.err.println(e); }
 } }
```


LA CLASSE DatagramSocket

Per la comunicazione senza connessione (UDP)

Creare una socket datagram:

```
DatagramSocket sock;
try {
    sock     =     new     DatagramSocket();
    }
    catch (SocketException e) {
       System.out.println(e.toString());
       e.printStackTrace();
       return;       }
```

NOTA: se da una socket datagram non dobbiamo ricevere ma solo spedire non importa specificare il numero di porta. Ce ne viene assegnato uno dal sistema

ATTENZIONE: se si specifica un numero di porta (in fase di creazione) bisogna assicurarsi che questo non sia già in uso o riservato

A questo punto la socket è già pronta per inviare/ricevere dati

Ma (IMPORTANTE!!!) siccome non c'è la connessione, non abbiamo neanche degli input-output stream!

E quindi? Dove mettiamo i messaggi da spedire

COMPORRE DATAGRAMMI PER LA CLASSE DatagramSocket

Un messaggio che vogliamo spedire dalla rete deve essere composto in uno stream da inviare.

Poiché la mancanza di connessione fa venire meno la presenza di uno stream sulla rete, creiamo un stream di byte local, cioè un array di byte, in cui comporre un messaggio (cioè i dati che faranno parte del datagramma)

```
// creiamo il Byte Array Output Stream
ByteArrayOutputStream baos;
baos =new ByteArrayOutputStream();
//e lo incapsuliamo nella classe di
//filtraggio che di fa piu' comodo
DataOutputStream dos;
try {
dos = new DataOutputStream(baos);
//poi scriviamo I dati da spedire nel
//nostro ByuteArrayOutputStream
oos.writeInt(3);
dos.writeFloat(4.56)
dos.close();  }
catch (IOException e) {return;}
```


(segue) COMPORRE DATAGRAMMI PER LA CLASSE DatagramSocket

```
// A questo punto mettiamo bytearray con
//i dati in un array di byte
byte[] data = baos.toByteArray();

// a questo punto creiamo il datagramma
// il datagramma, oltre ai dati, deve
// contenere l'indicazione del
// destinatario
InetAddress ia =
  InetAddress.getByName(www.test.com);
int port = 34 ;

DatagramPacket dp;
dp = new DatagramPacket(data, data.length,
ia, port);

try { // send è il servizio di spedizione
// di un datagramma
sock.send(dp);}
catch (IOException e) {}
```


RICEVERE DATAGRAMMI CON LA CLASSE DatagramSocket

Un messaggio che vogliamo recuperare dalla rete deve essere estratto dal datagramma che si riceve; In pratica, si fanno le operazioni di spedizione quasi "al contrario"

```
// dovremo avere una socket su cui ricevere
// dati su una specifica porta
// ATTENZIONE che la porta non sia in uso
int myPort = 34;
DatagramSocket ds =
      New DatagramSocket(myPort);
// creaiamo un array di byte
// abbastanza grande da poter contenere
// tutto il datagramma
int packetSize = 1000;
byte[] buf = new byte[packetSize];
// creaiamo il datagramma da ricevere
DatagramPacket dp =
   new DatagramPacket(buf, packetSize);
// servizio di ricezione di un datagramma
dgs.receive(dp);
// NOTA IMPORTANTE: la esecuzione si blocca
// fino a quando non arriva un datagramma!!
System.out.println("Datagram received");
```


(segue) RICEVERE DATAGRAMMI CON LA CLASSE DatagramSocket

```
// creaiamo un array di byte da cui estrarre
// i dati del messaggio del datagramma
// escludendo i dati mittente
byte[] data = dp.getData();

// creiamo un ByteArrayInputStream
// su cui depositare I dati
ByteArrayInputStream bais =
   new ByteArrayInputStream(data);

// lo incapsuliamo nella classe
// di filtraggio DataInputStream
// e poi leggiamo i dati
DataInputStream dis;
Int x; Float f;
try {
   dis = new ObjectInputStream(bais);
   x =  dis.readInt();
   f = dis.readFloat();
   dis.close(); }
catch (IOException e) {…}
```

URL E CONNESSIONI

- Per comunicare tramite http (o altri protocolli applicativi standard) in modo semplificato (ma meno flessibile) rispetto alle socket
- La classe `URL` cattura il concetto di *indirizzo Internet (URL)* nella forma standard del Web:
  - `http://localhost/index.html`
  - `file:///autoexec.bat`
  - `...`
- Un *oggetto* `URL` si crea a partire dall'indirizzo che rappresenta:

```
URL url = new URL("....");
```

e si usa per *aprire una connessione* verso tale indirizzo.

- Per aprire una connessione, si invoca sul-l'oggetto `URL` il metodo `openConnection()`:

```
URLConnection c =
              url.openConnection();
```

- Il risultato è un oggetto `URLConnection`, che rappresenta una "connessione aperta" con un server Web
  - in pratica, così facendo si è stabilito un canale di comunicazione verso l'indirizzo richiesto

URL E CONNESSIONI

- A differenza delle socket, il fatto di avere aperto la connessione non significa che ci si è veramente connessi (il protocollo 3-way handshaking di connessione non è stato ancora effettuato). Si è solo verificato che il server esista al sito specificato
- Per connettersi tramite tale connessione:

```
c.connect();
```

Per comunicare *si recuperano dalla connessione i due stream* (di ingresso e di uscita) a essa associati, tramite i metodi:

- `public InputStream getInputStream()`
  - restituisce lo stream di input da cui leggere i dati (byte) che giungono dall'altra parte
- `public OutputStream getOutputStream()`
  - restituisce lo stream di output su cui scrivere i dati (byte) da inviare all'altra parte

Poi, su questi stream si legge / scrive *come su qualunque altro stream di byte*.

ESEMPIO DI USO DI URL

Connettersi all'URL dato e, *nell'ipotesi che esso invii testo*, visualizzarne il contenuto

```
import java.io.*; import java.net.*;
class EsempioURL {
 public static void main(String args[]){
   URL u = null;
   try {
    u = new URL(args[0]);
//indirizzo di URL passato come argomento
   } catch (MalformedURLException e) {
     System.err.println("URL errato: " + u);
   }
   URLConnection c = null;
   try {
    System.out.print("Connecting...");
    c = u.openConnection(); c.connect();
    System.out.println("..OK");
    InputStreamReader is = new
      InputStreamReader(c.getInputStream());
    BufferedReader       r       =       new
   BufferedReader(is);
    System.out.println("Reading data...");
     String line = null;
     while((line=r.readLine())!=null)
       System.out.println(line);
   } catch (IOException e) {
     System.err.println(e);
   }  } }
```


ESEMPIO DI USO DI URL

Esempio invocazione e output programma precedente:

```
D:\esercizi>java EsempioURL file:///P.html

Connecting.....OK

Reading data...

<TITLE> Esempio di form </TITLE>
<H1> Esempio di form </H1>
<FORM                          METHOD="POST"
ACTION="http://localhost/cgi/prova.exe">
Inserisci il testo: <INPUT NAME="testo">
e  poi  premi  invio:  <INPUT  TYPE="submit"
VALUE="invio">
</FORM>
```

NOTA: Se avessimo la possibilitá di trasformare in forma grafica quello che il programma legge dall'URL avremmo praticamente realizzato un Browser Web.

Lo vedremo nel corso di programmazione Web….